General description: Molecular Dynamics Simulation package for the Rigid CCpol23+ Water Model Using Quaternion-Based Rigid Body Dynamics

Ommair Ishaque
ommair@udel.edu
Department of Physics and Astronomy, University of Delaware

May 1, 2026

Abstract

The CCpol23+ (water model) Molecular Dynamics (MD) Package is a Fortran 90/MPI code for classical simulations of liquid water using a first-principles, many-body force field. The model combines explicit two-body, three-body, and non-additive induction contributions that are derived from high-level electronic-structure benchmarks, with rigid water monomers represented via atomic and virtual interaction sites. The code implements quaternion-based rigid-body integration, long-range electrostatics through Ewald summation, and an induction model for many-body polarization. It is parallelized with MPI and is designed for medium- to large-scale simulations of bulk water and cluster systems. This document provides a technical description of the code architecture, main modules, algorithms, and typical usage patterns to support reproducible research and future extensions.

Contents

1 Introduction **3**

2 Overview of Code Structure **3**

3 Compilation and Build System **4**

4 Physical Models and Algorithms **5**

5 Module-by-Module Description **7**
5.1 `common_arrays.f90` . . . 7
5.2 `molecular_sites.f90` . . . 7
5.3 `utility.f90` . . . 8
5.4 `initialization.f90` . . . 8
5.5 `vdw_ccpol23plus.f90` . . . 8
5.6 `threebody_potential.f90` . . . 9
5.7 `nb_induction_model.f90` . . . 9
5.8 `ewald.f90` . . . 10
5.9 `evolve.f90` . . . 10
5.10 `statistics.f90` . . . 10
5.11 `md.f90` . . . 11

6 Simulation Workflow **11**

7 Example Usage **12**

8 Extensibility and Modifications **12**

9 Exchange repulsion, induction-dispersion and short range damped electrostatics terms in CCpol23+ **12**

10 Short-range three-body F_{S^3} interaction **14**

11 Ewald Decomposition for Charges and Induced Dipoles **16**
11.1 Local Electric Field 16
11.2 Ewald Decomposition 16
11.3 Real-Space Electric Field from Charges 16
11.4 Real-Space Electric Field from Dipoles 16
11.5 Reciprocal-Space Fields: Charges 16
11.6 Reciprocal-Space Fields: Dipoles 16
11.7 Self Terms 17

12 Induction Energy **17**

13 Self-Consistent Solution for Induced Dipoles **17**

14 Total Electrostatic + Induction Energy **17**

15 Force Expressions **17**
15.1 Charge Contribution 17
15.2 Dipole Contribution 18

16 Pseudocode for Force Evaluation **18**

17 How to Cite This Code **18**

1 Introduction

The MD Package was developed to perform classical molecular dynamics simulations of CCpol23+ water model [1] using a many-body potential that is fitted to coupled-cluster benchmark calculations. The force field decomposes the interaction energy into physically motivated components, including two-body short-range exchange–repulsion and dispersion, explicit three-body exchange and exchange–dispersion terms, and a non-additive induction model for many-body polarization. The present implementation focuses on rigid water monomers and is optimized for simulations of bulk liquid water, supercooled states, and cluster configurations.

The code is written in Fortran 90 with explicit modular structure. Each physical contribution (two-body, three-body, induction, electrostatics) is implemented in a separate module, which is coupled to the rigid-body equations of motion through a central integration module. Global data structures and system-wide parameters are collected in dedicated array and topology modules. The main driver coordinates initialization, time integration, sampling, and MPI parallelization.

This document is intended as a software reference manual for users who wish to run new simulations, analyze output, or extend the code. It focuses on describing the responsibilities of each module, the high-level algorithms implemented, the flow of data through the code, and the practical steps needed to compile and execute the program. Detailed force-field derivations and fitting procedures are assumed to be documented in accompanying scientific articles.

2 Overview of Code Structure

The source directory for the CCpol23+ MD code is organized as a single collection of Fortran 90 files together with a build system. The core modules at the time of this documentation are:

- `common_arrays.f90`: global arrays, constants, and shared variables that describe the system size, simulation cell, and thermodynamic state.
- `molecular_sites.f90`: definitions of the water monomer geometry, including atomic and virtual sites, charges, and other site-specific parameters.
- `utility.f90`: general-purpose helper routines used across the code base, such as input parsing utilities, safe allocation and error-checking helpers, vector and tensor operations, and small mathematical or geometric utilities.
- `initialization.f90`: routines for reading input, allocating arrays, constructing the initial configuration, and assigning initial velocities or orientations.
- `vdw_ccpol23plus.f90`: implementation of the two-body van der Waals interaction model used in the CCpol23+ force field.
- `threebody_potential.f90`: implementation of the explicit three-body term (FS3), including short-range three-body exchange and exchange–dispersion contributions.
- `nb_induction_model.f90`: non-additive many-body induction model with self-consistent induced dipoles or induced field degrees of freedom.
- `ewald.f90`: routines for reciprocal-space and real-space electrostatic and polarization contributions, including Ewald summation for charges and dipoles.
- `evolve.f90`: time-integration routines implementing quaternion-based rigid-body dynamics and updating atomic and virtual site positions.
- `statistics.f90`: collection of analysis and sampling routines for energies, pressure, density, and other observables.

- `md.f90`: main program coordinating initialization, force evaluation, integration, sampling, and MPI communication.
- `Makefile`: build system specifying source file dependencies, compiler and linker flags, and convenience targets for compiling and cleaning the code.

A typical build command on a Unix-like system with MPI support is:

```
mpif90 -O3 -o md_ccpol23plus.x \
    common_arrays.f90        \
    molecular_sites.f90      \
    utility.f90              \
    initialization.f90       \
    vdw_ccpol23plus.f90      \
    threebody_potential.f90  \
    nb_induction_model.f90   \
    ewald.f90                \
    evolve.f90               \
    statistics.f90           \
    md.f90
```

When a `Makefile` is provided, the code can instead be compiled simply as:

```
make
```

producing an executable such as `md_ccpol23plus.x` according to the settings defined in the `Makefile`.

Execution on four MPI ranks proceeds, for example, as:

```
mpirun -np 4 ./md_ccpol23plus.x
```

The exact name of the executable and input file may be adapted to project-specific conventions.

3 Compilation and Build System

The CCpol23+ MD code is written in standard-conforming Fortran 90 and is intended to be compiled with a modern Fortran compiler that supports at least the following features:

- Modules and explicit interfaces.
- Allocatable arrays and derived types.
- Double-precision floating point arithmetic.

For parallel execution, an MPI implementation (e.g., MPICH, Open MPI, or an HPC vendor MPI library) is required. The main program and several modules use standard MPI calls for domain decomposition or parallel summation of energies and forces.

Makefile

The provided `Makefile` encapsulates the compilation procedure and ensures that modules are built in a correct order. Typical features of the `Makefile` include:

- Definitions of the Fortran compiler (`FC`) and flags for production and debug builds (e.g., optimization, array bounds checking).

- A list of all source files (`common_arrays.f90`, `molecular_sites.f90`, `utility.f90`, `initialization.f90`, `vdw_ccpol23plus.f90`, `threebody_potential.f90`, `nb_induction_model.f90`, `ewald.f90`, `evolve.f90`, `statistics.f90`, and `md.f90`) and the corresponding object files.
- A default target (e.g., `all`) that builds the CCpol23+ MD executable.
- A `clean` target for removing object files, module files, and the executable to allow a fresh rebuild.

Using the `Makefile` is recommended, as it automatically tracks dependencies between modules and simplifies switching between different compilers or architectures.

No external libraries are required beyond MPI. The induction model, three-body potential, and Ewald summations are implemented entirely in Fortran within this code base.

4 Physical Models and Algorithms

Rigid water monomers

The code treats each water molecule as a rigid body with a fixed internal geometry. The monomer geometry and site definitions are contained in `molecular_sites.f90`. Each molecule is described by:

- Atomic positions in the body-fixed frame (O and H atoms).
- One or more virtual or off-center sites used to represent charge distribution or polarizable sites.
- Site charges and, where applicable, polarizabilities or other site-specific parameters.

Rigid-body motion is propagated via quaternions, with each molecule having a center-of-mass position and a unit quaternion describing its orientation. Forces and torques are computed in the laboratory frame and mapped to translational and rotational degrees of freedom.

Two-body interactions

Two-body interactions are implemented in `vdw_ccpol23plus.f90`. The module evaluates pairwise contributions between sites on different molecules, including:

- Short-range exchange–repulsion and dispersion between atomic and virtual sites, typically represented by analytic functions calibrated to high-level ab initio data.
- Long-range asymptotic behavior consistent with the multipole expansion of the interaction energy.
- Short-range damping or switching functions that regularize the interactions at small intermolecular separations.

The two-body module supplies both energy and analytic forces between molecules, as required by the MD integrator.

Three-body interactions

The explicit three-body term is implemented in `threebody_potential.f90`. This module evaluates a three-body exchange and exchange–dispersion potential for all water trimers within a specified three-body cutoff. The functional form is typically expressed in terms of pairwise distances between monomers and incorporates:

- A short-range region described by a flexible analytic fit to three-body ab initio energies.
- Long-range damping functions that ensure a smooth decay toward the asymptotic three-body limit.
- A switching scheme that connects short- and long-range regimes in a smooth and energy-conserving manner.

The three-body module returns both energies and forces with respect to atomic or site coordinates, which are subsequently accumulated into molecular forces and torques.

Many-body induction model

Non-additive many-body polarization is handled by the induction module `nb_induction_model.f90`. The general structure is:

- Each polarizable site is assigned a polarizability and, if applicable, a damping parameter.
- Local electric fields are constructed from permanent multipoles and induced dipoles.
- A self-consistent field (SCF) procedure iteratively updates the induced dipoles until convergence, subject to damping functions that prevent unphysical polarization at short range.

The module typically provides:

- Subroutines to compute the induced dipoles from a given configuration of permanent charges and multipoles.
- Routines to accumulate the induction energy and its gradients, which contribute to forces and torques.
- Interfaces to the Ewald module to account for long-range polarization under periodic boundary conditions.

Electrostatics and Ewald summation

The `ewald.f90` module implements Ewald summation for electrostatic interactions and, where needed, for dipole–dipole and charge–dipole terms. The key tasks include:

- Real-space contributions between charges and polarization sites within a cutoff.
- Reciprocal-space sums over a set of reciprocal lattice vectors, including structure factors and Fourier-space fields.
- Self-interaction corrections, background terms, and virial contributions consistent with periodic boundary conditions.

The Ewald routines are called during force evaluation and return contributions to both energy and forces.

Time integration

Time propagation of the rigid-body degrees of freedom is performed in `evolve.f90`. The main features are:

- Rigid-body translational motion integrated using a standard velocity-Verlet or related symplectic scheme.
- Rotational motion of each molecule represented via unit quaternions, with torques mapped to angular momenta and quaternion updates ensuring normalization.
- Consistent update of virtual site positions derived from the underlying rigid-body geometry and current quaternion.

The integration module is designed to work with forces and torques provided by the two-body, three-body, induction, and Ewald modules, and may support NVE and, optionally, thermostatted or barostatted ensembles depending on the specific implementation.

5 Module-by-Module Description

This section summarizes the roles of each source file and its principal routines. The intention is to give a conceptual overview rather than a line-by-line API reference. Exact routine names and argument lists can be inspected directly in the source.

5.1 `common_arrays.f90`

The `common_arrays` module collects global arrays and parameters that describe the system and simulation state. Typical contents include:

- Numbers of molecules, sites per molecule, and total sites.
- Simulation cell vectors, inverse cell, and volume.
- Global arrays for positions, velocities, forces, and torques for each site or each molecule.
- Thermodynamic state variables such as target temperature, time step, and total number of integration steps.
- Constants such as the Boltzmann constant, elementary charge, conversion factors between internal units and SI or laboratory units.

This module is used by nearly all other parts of the program to share common data in a controlled manner.

5.2 `molecular_sites.f90`

The `molecular_sites` module defines the internal structure of a rigid water monomer and the mapping between molecular and site-level quantities. Typical responsibilities are:

- Storing the body-fixed positions of O and H atoms in the monomer frame.
- Defining virtual or off-center sites (e.g., for representing lone pairs or effective charge locations).
- Assigning site charges, polarizabilities, and possibly other site-specific parameters used by the force field.

- Providing routines to construct laboratory-frame site positions from a given center-of-mass position and orientation quaternion.

This module forms the bridge between high-level molecular coordinates and the site-based evaluation of energies and forces.

5.3 `utility.f90`

The `utility` module contains general-purpose helper routines that are used throughout the code base. While the exact content can evolve with the code, typical categories of functionality include:

- Input and string-handling utilities, such as parsing simple keyword–value pairs from text files or cleaning and splitting lines.
- Safe allocation and error-checking wrappers that centralize checks for allocation failures or invalid array dimensions.
- Small vector and tensor operations that are not covered by the main physics modules, such as dot products, norms, or simple rotations.
- Utility routines for timing and diagnostics, for example to measure wall-clock time for different sections of the code.

By centralizing such helper routines in a single module, the main physics and integration modules remain focused on their primary tasks, and code reuse is facilitated.

5.4 `initialization.f90`

The `initialization` module handles all tasks associated with setting up a simulation:

- Reading the main input file, including system size, temperature, density or box dimensions, time step, cutoffs, and ensemble control parameters.
- Allocating arrays in `common_arrays` and other modules according to the requested number of molecules and sites.
- Setting up the initial molecular configuration, either by placing molecules on a lattice, reading a pre-equilibrated configuration, or constructing a cluster geometry.
- Assigning initial velocities consistent with the target temperature, for example using a Maxwell–Boltzmann distribution and removing center-of-mass drift.
- Initializing random number seeds and any bookkeeping data structures.

After initialization completes, the system is ready for the first force evaluation and MD step.

5.5 `vdw_ccpol23plus.f90`

The `vdw_ccpol23plus` module implements the two-body part of the CCpol23+ potential. At a high level, it provides routines such as "compute two-body energy and forces," which:

- Loop over molecule pairs within the chosen two-body cutoff.
- For each pair, loop over relevant site pairs and evaluate the short-range van der Waals interaction, including exchange and dispersion.

- Apply damping or switching functions that regularize the interaction in the short-range region and ensure a smooth approach to the long-range behavior.
- Accumulate pair contributions into the global energy and force arrays.

The precise functional form and parameter values are inherited from the CCpol23+ fitting procedure and are assumed to be fixed for a given potential version.

5.6 `threebody_potential.f90`

The `threebody_potential` module evaluates the explicit three-body term in the force field. Its central tasks comprise:

- Construction of trimer lists by looping over triplets of molecules with pair distances below the specified three-body cutoff.
- Evaluation of the three-body exchange and exchange–dispersion energy for each trimer, using analytic expressions in terms of inter-monomer distances and, where relevant, intra-monomer geometrical parameters.
- Application of three-body damping and switching functions that control the range of the three-body term and its connection to the underlying many-body expansion.
- Computation of analytic gradients of the three-body energy with respect to site coordinates and accumulation of resulting forces and torques.

This module is responsible for capturing the non-additive three-body effects that are essential for accurate modeling of water.

5.7 `nb_induction_model.f90`

The `nb_induction_model` module implements the many-body induction model used in CCpol23+. In a typical SCF induction cycle:

- Permanent fields at each polarizable site are computed based on the current configuration of permanent charges, multipoles, and, where applicable, other interaction sites.
- Induced dipoles (or analogous induced quantities) are updated using the local field and site polarizabilities, subject to damping functions that prevent unphysical polarization at short range.
- The induced field contribution is added to the total field and the update is iterated until convergence, potentially with under-relaxation or other convergence accelerators.
- Once converged, the induction energy and the force contributions associated with the induced dipoles are evaluated and accumulated.

The module typically also provides interfaces to the Ewald routines to treat the long-range part of the induction interactions under periodic boundary conditions.

5.8 `ewald.f90`

The `ewald` module is responsible for the Ewald summation of electrostatic and polarization interactions. Its main components include:

- Real-space pair interactions between charges and dipoles within a specified cutoff, including damping functions and near-field corrections as required.
- Reciprocal-space contributions computed by summing over reciprocal lattice vectors. Structure factors are built from the permanent and induced site properties, and Fourier-space fields are used to accumulate energies, forces, and virial contributions.
- Self-energy and background terms required for consistency with the chosen Ewald convention.

The Ewald module is called during each force evaluation to ensure that long-range electrostatics and polarization are treated accurately.

5.9 `evolve.f90`

The `evolve` module contains the time-integration routines for rigid-body dynamics. At a conceptual level, the integration loop performs the following operations each time step:

- Update molecular velocities and angular momenta using the forces and torques from the previous step.
- Propagate molecular positions and orientations forward in time, with rotational updates represented via quaternion operations that preserve the unit norm constraint.
- Rebuild site positions for all atomic and virtual sites from the updated center-of-mass positions and quaternions.
- Apply periodic boundary conditions to keep molecules within the primary simulation cell and maintain translational invariance.
- Integrators are implemented using the NOvel Symplectic QUaternIon ScHeme (NOSQUISH) algorithm [2, 3]. This algorithm is fully symplectic, does not required any iterations, and is stable. In this method equations of motion for rigid bodies are represented in the Hamiltonian representation using quaternion algebra to enforce time reversibility and stability for larger time steps. Due to feature of this type methods are known as sympletic and reversible integrators and are commonly used in MD.

Depending on configuration, the module may also interface with thermostats or barostats, though rigid-body NVE integration is the core capability but it also has NPT and NVT implementations.

5.10 `statistics.f90`

The `statistics` module provides utilities for monitoring and analyzing the simulation. Typical responsibilities include:

- Accumulation of kinetic, potential, and total energies, as well as pressure, virial, and other thermodynamic quantities.
- Online averaging of observables over time and block averaging for statistical error estimation.

- Construction of radial distribution functions and other structural observables via histogramming of pair distances.
- Output of trajectory snapshots, energy logs, and diagnostic information at user-specified intervals.

These routines are invoked regularly from the main MD driver to produce data suitable for later analysis and publication.

5.11 `md.f90`

Finally, `md.f90` contains the main program and overall control flow. At a high level, it performs:

1. Initialization of MPI and determination of the rank and size.
2. Reading of the input file and calling initialization routines to allocate arrays and build the initial configuration.
3. Entry into the MD loop, which typically consists of:
   3.1. Force evaluation: calling the two-body, three-body, induction, and Ewald modules to compute total forces and torques.
   3.2. Time step integration: calling the `evolve` module to update positions and orientations.
   3.3. Application of boundary conditions and any constraint handling.
   3.4. Sampling and output via `statistics`.
4. Finalization of MPI and closure of all output streams.

The main program therefore acts as an orchestrator that couples all physical models into a consistent MD simulation.

6 Simulation Workflow

A typical CCpol23+ MD simulation proceeds as follows:

1. **Prepare an input file** specifying the number of water molecules, target thermodynamic state, simulation box or density, time step, cutoffs for two-body and three-body interactions, and output frequencies.
2. **Compile the code** using the provided `Makefile` or a manual compilation command, producing an executable such as `md_ccpol23plus.x`.
3. **Run the code** with `mpirun` or an equivalent launcher, providing the input file name on the command line.
4. **Monitor the simulation** via the energy log and any periodic printed diagnostics (e.g., temperature, density, pressure).
5. **Post-process the output** using analysis scripts to compute observables such as radial distribution functions, density and diffusivity as a function of temperature, or cluster energetics.

The exact input syntax, output file names, and analysis scripts may be customized by the user; this manual focuses on the core MD engine.

7 Example Usage

An example input file for a bulk liquid water simulation might specify:

- Number of molecules (e.g., 256).
- Initial density or box length.
- Target temperature and time step.
- Cutoffs for two-body (e.g., 9.0 Å) and three-body terms (e.g., 6.5–7.0 Å).
- Length of the equilibration and production runs.
- Frequencies for printing energies, configurations, and structural data.

The user may then perform a temperature series, for example, to compute the density and structural properties over a range of temperatures and compare to experimental data or to other water models. Cluster calculations can be set up similarly, with appropriately chosen initial geometries. See attached Readme file and example folder.

8 Extensibility and Modifications

The modular design of the CCpol23+ MD code makes it straightforward to extend or modify specific parts of the force field or simulation algorithm. Examples include:

- Replacing the two-body module `vdw_ccpol23plus.f90` with an alternative pair potential, while keeping the induction and three-body models fixed.
- Introducing alternative three-body forms in `threebody_potential.f90`, for example polynomial expansions or machine-learning-based fits.
- Modifying the induction model in `nb_induction_model.f90` to test different damping schemes or convergence accelerators.
- Extending `statistics.f90` with additional structural or dynamical observables tailored to a given research question.

Care should be taken to preserve energy conservation and consistency between energy and force expressions whenever the potential energy surface is modified.

9 Exchange repulsion, induction-dispersion and short range damped electrostatics terms in CCpol23+

For a pair of molecules I, J and van der Waals sites $s \in I$, $t \in J$, we define the site–site separation

$$\mathbf{r}_{st} = \mathbf{R}_I - \mathbf{R}_J + \mathbf{r}_{Is} - \mathbf{r}_{Jt}, \qquad r \equiv r_{st} = \|\mathbf{r}_{st}\|, \tag{1}$$

using the minimum-image convention for $\mathbf{R}_I - \mathbf{R}_J$. For each site-type pair k we use:

Short-range exchange–repulsion with logistic damping

$$P^{(k)}(r) = c_0^{(k)} + c_1^{(k)} r + c_2^{(k)} r^2 + c_3^{(k)} r^3, \tag{2}$$

$$f_{\mathrm{L}}(r) = \frac{1}{1 + \exp\big[-\gamma(r - r_0)\big]}, \tag{3}$$

$$V_{\exp}^{(k)}(r) = e^{-\beta_k r}\, P^{(k)}(r)\, f_{\mathrm{L}}(r). \tag{4}$$

Tang–Toennies damping The Tang–Toennies damping function of order n with parameter d is

$$f_n(d,r) = 1 - e^{-dr} \sum_{m=0}^{n} \frac{(dr)^m}{m!}, \tag{5}$$

with derivative $\partial f_n/\partial r$ evaluated analytically in the code.

Short-range electrostatic correction For site charges q_i, q_j and $d_1^{(k)}$,

$$V_{\mathrm{el}}^{(k)}(r) = \frac{1}{4\pi\varepsilon_0} \frac{\left[f_1(d_1^{(k)}, r) - 1\right] q_i q_j}{r}. \tag{6}$$

Damped dispersion/induction tail With dispersion/induction coefficients $C_6^{(k)}, C_8^{(k)}, C_{10}^{(k)}$ and damping parameters $d_6^{(k)}, d_8^{(k)}, d_{10}^{(k)}$,

$$V_{\mathrm{ind-disp}}^{(k)}(r) = -\frac{C_6^{(k)} f_6(d_6^{(k)}, r)}{r^6} - \frac{C_8^{(k)} f_8(d_8^{(k)}, r)}{r^8} - \frac{C_{10}^{(k)} f_{10}(d_{10}^{(k)}, r)}{r^{10}}. \tag{7}$$

Total short-range site–site interaction

$$V_{st}(r) = V_{\mathrm{exp}}^{(k)}(r) + V_{\mathrm{el}}^{(k)}(r) + V_{\mathrm{disp}}^{(k)}(r), \tag{8}$$

where k is the parameter index corresponding to the site-type pair (s,t).

Force and virial The radial force magnitude is obtained from

$$\frac{\mathrm{d}V_{st}}{\mathrm{d}r} = \frac{\mathrm{d}V_{\mathrm{exp}}^{(k)}}{\mathrm{d}r} + \frac{\mathrm{d}V_{\mathrm{el}}^{(k)}}{\mathrm{d}r} + \frac{\mathrm{d}V_{\mathrm{ind-disp}}^{(k)}}{\mathrm{d}r}, \tag{9}$$

and the vector force and pair virial are

$$\mathbf{F}_{st} = -\frac{\mathrm{d}V_{st}}{\mathrm{d}r} \frac{\mathbf{r}_{st}}{r}, \qquad \Xi_{st} = -\mathbf{r}_{st} \cdot \mathbf{F}_{st}. \tag{10}$$

Long-range corrections Beyond the cutoff r_c, assuming $g(r) \approx 1$, the analytic energy and virial corrections are of the form

$$U_{\mathrm{LRC}} = 2\pi\rho^2 \sum_k \int_{r_c}^{\infty} V^{(k)}(r)\, r^2\, \mathrm{d}r, \tag{11}$$

$$W_{\mathrm{LRC}} = -\frac{2\pi\rho^2}{3} \sum_k \int_{r_c}^{\infty} r^3 \frac{\mathrm{d}V^{(k)}(r)}{\mathrm{d}r}\, \mathrm{d}r, \tag{12}$$

where $\rho = n/V$ is the number density. In the implementation, the integrals in Eqs. (11)–(12) are evaluated analytically for the specific functional form (4)–(7), and then scaled by $2\pi n^2/V$.

Algorithm 1 Evaluation of CCpol23+ van der Waals Term with Long-Range Correction

Require: Molecular centers $\{\mathbf{R}_I\}$, site offsets $\{\mathbf{r}_{Is}\}$
Require: Site charges q_i, vdW parameters $\{c_m^{(k)}, \beta_k, C_6^{(k)}, C_8^{(k)}, C_{10}^{(k)}, d_1^{(k)}, d_6^{(k)}, d_8^{(k)}, d_{10}^{(k)}\}$
Require: Logistic damping parameters γ, r_0, cutoffs r_{cut}, r_c
Ensure: Short-range vdW energy U_{vdW}, forces, virial, and LRC corrections

```
U_vdW ← 0, initialize all forces and virial tensor to zero
for each molecular pair (I, J) do
    Compute COM separation R_IJ (minimum image)
    if ‖R_IJ‖ < r_cut then
        for each site pair (s, t) on (I, J) do
            Build r_st and r = ‖r_st‖
            Identify parameter index k for the site-type pair (s, t)
            Evaluate V_exp^(k)(r) using Eq. (4)
            Evaluate V_el^(k)(r) using Eq. (6)
            Evaluate V_ind−disp^(k)(r) using Eq. (7)
            V_st(r) ← V_exp^(k) + V_el^(k) + V_disp^(k)
            U_vdW ← U_vdW + V_st(r)
            Evaluate dV_st/dr (analytic derivative of the above)
            Compute F_st and Ξ_st via Eq. (10)
            Accumulate forces on sites s and t, and add Ξ_st to the virial
        end for
    end if
end for
Long-range correction:
For each parameter set k, evaluate analytic contributions to U_LRC and W_LRC using
Eqs. (11)–(12) with V^(k)(r) from Eq. (8)
Add U_LRC to U_vdW and include W_LRC in the virial
return U_vdW (including LRC), forces, and virial tensor
```

10 Short-range three-body F_{S^3} interaction

For a triplet of molecules (I, J, K) with centers of mass (COM) at $\mathbf{R}_I, \mathbf{R}_J, \mathbf{R}_K$, we define the COM separation vectors (with minimum-image convention)

$$\mathbf{R}_{IJ} = \mathbf{R}_I - \mathbf{R}_J, \quad \mathbf{R}_{IK} = \mathbf{R}_I - \mathbf{R}_K, \quad \mathbf{R}_{JK} = \mathbf{R}_J - \mathbf{R}_K, \tag{13}$$

with corresponding distances

$$R_{IJ} = \|\mathbf{R}_{IJ}\|, \qquad R_{IK} = \|\mathbf{R}_{IK}\|, \qquad R_{JK} = \|\mathbf{R}_{JK}\|. \tag{14}$$

At the site level, for sites $a \in I$, $b \in J$, $c \in K$ we define

$$\mathbf{r}_{ab} = \mathbf{R}_{IJ} + \mathbf{r}_{Ia} - \mathbf{r}_{Jb}, \qquad r_{ab} = \|\mathbf{r}_{ab}\|, \tag{15}$$

$$\mathbf{r}_{ac} = \mathbf{R}_{IK} + \mathbf{r}_{Ia} - \mathbf{r}_{Kc}, \qquad r_{ac} = \|\mathbf{r}_{ac}\|, \tag{16}$$

$$\mathbf{r}_{bc} = \mathbf{R}_{JK} + \mathbf{r}_{Jb} - \mathbf{r}_{Kc}, \qquad r_{bc} = \|\mathbf{r}_{bc}\|. \tag{17}$$

Three-body COM damping A smooth three-body cutoff is imposed at the COM level via a logistic damping function for each COM pair,

$$f_3(R) = \left[1 + \exp\big(\alpha_3(R - \tilde{r}_0)\big)\right]^{-\delta_3}, \tag{18}$$

so that the total COM damping factor for the trimer (I, J, K) is

$$F_{\mathrm{COM}}(R_{IJ}, R_{IK}, R_{JK}) = f_3(R_{IJ})\, f_3(R_{IK})\, f_3(R_{JK}). \tag{19}$$

The radial derivative $\partial f_3/\partial R$ is evaluated analytically in the code.

Site-level short-range factors For each site-type pair (a, b) we define a short-range, damped exponential

$$\lambda_{ab}(r) = \frac{1}{1 + \exp\left[-g_{ab}(r - r_{0,ab})\right]}, \tag{20}$$

$$f_{ab}(r) = \exp(-b_{ab} r)\, \lambda_{ab}(r), \tag{21}$$

with parameters b_{ab}, g_{ab} and $r_{0,ab}$. The corresponding radial derivative is

$$\frac{\mathrm{d}f_{ab}}{\mathrm{d}r} = f_{ab}(r) \left[-b_{ab} + g_{ab}\, e^{-g_{ab}(r - r_{0,ab})}\, \lambda_{ab}(r)\right], \tag{22}$$

which is used to construct analytic forces.

For a triplet of sites (a, b, c) belonging to (I, J, K), we define the product

$$f_{abc}(r_{ab}, r_{ac}, r_{bc}) = f_{ab}(r_{ab})\, f_{ac}(r_{ac})\, f_{bc}(r_{bc}). \tag{23}$$

Polynomial basis in the three site–site distances For each site-type triplet (τ_a, τ_b, τ_c) and integer exponents $k_{\mathrm{AB}}, k_{\mathrm{AC}}, k_{\mathrm{BC}} \in \{0, \ldots, l_{\max}\}$, we define a monomial

$$M^{(\tau_a \tau_b \tau_c)}_{k_{\mathrm{AB}}, k_{\mathrm{AC}}, k_{\mathrm{BC}}}(r_{ab}, r_{ac}, r_{bc}) = r_{ab}^{k_{\mathrm{AB}}}\, r_{ac}^{k_{\mathrm{AC}}}\, r_{bc}^{k_{\mathrm{BC}}}. \tag{24}$$

Each symmetry-unique combination is assigned a linear index

$$\nu = \mathrm{ind}(\tau_a, \tau_b, \tau_c, k_{\mathrm{AB}}, k_{\mathrm{AC}}, k_{\mathrm{BC}}), \tag{25}$$

with associated coefficient c_ν. The site-level three-body interaction for (a, b, c) is then

$$\Phi_{abc}(r_{ab}, r_{ac}, r_{bc}) = \sum_{k_{\mathrm{AB}}=0}^{l_{\max}} \sum_{k_{\mathrm{AC}}=0}^{l_{\max}} \sum_{k_{\mathrm{BC}}=0}^{l_{\max}} c_{\mathrm{ind}(\tau_a, \tau_b, \tau_c, k_{\mathrm{AB}}, k_{\mathrm{AC}}, k_{\mathrm{BC}})}\, f_{abc}\, M^{(\tau_a \tau_b \tau_c)}_{k_{\mathrm{AB}}, k_{\mathrm{AC}}, k_{\mathrm{BC}}}. \tag{26}$$

Total FS^3 three-body potential The short-range three-body F_{S^3} potential is a sum over all distinct molecular trimers and site triplets,

$$V_{F_{S^3}} = \sum_{I<J<K} F_{\mathrm{COM}}(R_{IJ}, R_{IK}, R_{JK}) \sum_{a \in I} \sum_{b \in J} \sum_{c \in K} \Phi_{abc}(r_{ab}, r_{ac}, r_{bc}), \tag{27}$$

with an implicit three-body COM cutoff: V_{FS^3} vanishes whenever any of R_{IJ}, R_{IK} or R_{JK} exceeds the cutoff distance.

Forces and virial For a fixed trimer (I, J, K) and site triplet (a, b, c), the F_{S^3} contribution to the force on site a is obtained by the chain rule

$$\mathbf{F}_a^{(F_{S^3})} = -\nabla_{\mathbf{r}_a} V_{\mathrm{FS}^3} = -F_{\mathrm{COM}} \left[\frac{\partial \Phi_{abc}}{\partial r_{ab}} \frac{\mathbf{r}_{ab}}{r_{ab}} + \frac{\partial \Phi_{abc}}{\partial r_{ac}} \frac{\mathbf{r}_{ac}}{r_{ac}}\right] + \text{COM-damping contributions}, \tag{28}$$

and similarly for sites b and c. The derivatives $\partial \Phi_{abc}/\partial r_{ab}$, etc., are computed analytically from Eq. (26) using Eqs. (22)–(23). Additional force contributions arise from the gradients of the COM damping factor F_{COM} in Eq. (19), which act on the molecular COM coordinates.

The three-body F_{S^3} contribution to the scalar virial is

$$\Xi_{F_{S^3}} = -\frac{1}{3} \sum_n \mathbf{r}_n \cdot \mathbf{F}_n^{(F_{S^3})}, \tag{29}$$

where the sum runs over all sites and molecular centers affected by the FS^3 interaction. In the implementation, the corresponding Cartesian components are accumulated explicitly and then converted to the desired pressure units.

11 Ewald Decomposition for Charges and Induced Dipoles

Consider a system of N polarizable sites with positions $\mathbf{r}_i$, charges q_i, polarizabilities α_i, and induced dipoles $\boldsymbol{\mu}_i$.

11.1 Local Electric Field

The local electric field at site i is

$$\mathbf{E}_i^{\text{loc}} = \mathbf{E}_i^{(q)} + \mathbf{E}_i^{(\mu)}. \tag{30}$$

Induced dipoles satisfy

$$\boldsymbol{\mu}_i = \alpha_i \, \mathbf{E}_i^{\text{loc}}. \tag{31}$$

11.2 Ewald Decomposition

For both charges and dipoles, the Ewald splitting is

$$\mathbf{E}_i^{(q)} = \mathbf{E}_{i,\text{real}}^{(q)} + \mathbf{E}_{i,\text{k}}^{(q)} + \mathbf{E}_{i,\text{self}}^{(q)}, \tag{32}$$

$$\mathbf{E}_i^{(\mu)} = \mathbf{E}_{i,\text{real}}^{(\mu)} + \mathbf{E}_{i,\text{k}}^{(\mu)} + \mathbf{E}_{i,\text{self}}^{(\mu)}. \tag{33}$$

Here α is the Ewald screening parameter.

11.3 Real-Space Electric Field from Charges

$$\mathbf{E}_{i,\text{real}}^{(q)} = \sum_{j \neq i} q_j \frac{\operatorname{erfc}(\alpha r_{ij})}{r_{ij}^3} \, \mathbf{r}_{ij}, \tag{34}$$

where $\mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j$.

11.4 Real-Space Electric Field from Dipoles

$$\mathbf{E}_{i,\text{real}}^{(\mu)} = \sum_{j \neq i} \mathbf{T}_{ij}^{\text{real}}(\alpha, r_{ij}) \, \boldsymbol{\mu}_j, \tag{35}$$

where $\mathbf{T}^{\text{real}}$ is the Ewald-damped dipole–dipole tensor.

11.5 Reciprocal-Space Fields: Charges

Define the charge structure factor

$$S_q(\mathbf{G}) = \sum_j q_j e^{i\mathbf{G}\cdot\mathbf{r}_j}. \tag{36}$$

The reciprocal-space electric field is

$$\mathbf{E}_{i,\text{k}}^{(q)} = \frac{4\pi}{V} \sum_{\mathbf{G}\neq 0} \frac{e^{-G^2/(4\alpha^2)}}{G^2} \, S_q(\mathbf{G}) \, i\mathbf{G} e^{-i\mathbf{G}\cdot\mathbf{r}_i}. \tag{37}$$

11.6 Reciprocal-Space Fields: Dipoles

Define the dipole structure factor

$$\mathbf{S}_\mu(\mathbf{G}) = \sum_j \boldsymbol{\mu}_j e^{i\mathbf{G}\cdot\mathbf{r}_j}. \tag{38}$$

The k-space electric field from dipoles is

$$\mathbf{E}_{i,\text{k}}^{(\mu)} = \frac{4\pi}{V} \sum_{\mathbf{G}\neq 0} e^{-G^2/(4\alpha^2)} \left[\frac{1}{G^2} \left(\mathbb{I} - \frac{\mathbf{G}\mathbf{G}}{G^2} \right) \mathbf{S}_\mu(\mathbf{G}) \right] e^{-i\mathbf{G}\cdot\mathbf{r}_i}. \tag{39}$$

11.7 Self Terms

$$\mathbf{E}_{i,\text{self}}^{(q)} = C_q(q_i, \alpha), \tag{40}$$

$$\mathbf{E}_{i,\text{self}}^{(\mu)} = C_\mu(\boldsymbol{\mu}_i, \alpha), \tag{41}$$

where C_q and C_μ are analytic Ewald self-corrections.

12 Induction Energy

$$E_{\text{ind}} = -\frac{1}{2}\sum_i \boldsymbol{\mu}_i \cdot \mathbf{E}_i^{\text{loc}}. \tag{42}$$

13 Self-Consistent Solution for Induced Dipoles

Algorithm 2 Self-Consistent Field (SCF) for Induced Dipoles

Initialize $\boldsymbol{\mu}_i^{(0)} = 0$.
for $n = 1$ to max_iter **do**
  Compute $\mathbf{E}_{i,\text{real}}^{(q)}$ and $\mathbf{E}_{i,\text{real}}^{(\mu)}$.
  Compute $\mathbf{E}_{i,\text{k}}^{(q)}$ and $\mathbf{E}_{i,\text{k}}^{(\mu)}$.
  Add self terms: $\mathbf{E}_{i,\text{self}}^{(q)}$, $\mathbf{E}_{i,\text{self}}^{(\mu)}$.
  Form the total local field:

$$\mathbf{E}_i^{(n)} = \mathbf{E}_i^{(q)} + \mathbf{E}_i^{(\mu)}.$$

  Update induced dipoles:

$$\boldsymbol{\mu}_i^{(n)} = \alpha_i \mathbf{E}_i^{(n)}.$$

  if $\max_i \left|\boldsymbol{\mu}_i^{(n)} - \boldsymbol{\mu}_i^{(n-1)}\right| < \varepsilon$ **then**
    Converged.
    break
  end if
end for

14 Total Electrostatic + Induction Energy

$$E_{\text{tot}} = E_{qq} + E_{q\mu} + E_{\mu\mu} + E_{\text{ind}}, \tag{43}$$

where each term contains real-space, reciprocal-space, and self components.

15 Force Expressions

The total force on site i is

$$\mathbf{F}_i = -\frac{\partial E_{\text{tot}}}{\partial \mathbf{r}_i}. \tag{44}$$

15.1 Charge Contribution

$$\mathbf{F}_i^{(q)} = q_i\, \mathbf{E}_i^{\text{tot}}. \tag{45}$$

15.2 Dipole Contribution

$$\mathbf{F}_i^{(\mu)} = -\frac{\partial}{\partial \mathbf{r}_i}\left(\sum_j \tfrac{1}{2}\boldsymbol{\mu}_j \cdot \mathbf{E}_j^{\mathrm{loc}}\right), \tag{46}$$

which is evaluated via real-space and reciprocal-space derivatives of the Ewald kernels.

16 Pseudocode for Force Evaluation

Algorithm 3 Force Computation (Real + k-Space + Self)

```
Initialize all forces F_i = 0.
for all pairs (i, j) in real space do
    Compute qq force:
        F_ij^(qq).
    Compute charge–dipole force:
        F_ij^(qμ).
    Compute dipole–dipole force:
        F_ij^(μμ).
    Accumulate:
        F_i += F_ij,    F_j -= F_ij.
end for
for all G in reciprocal space do
    Evaluate derivatives of E_qq, E_qμ, E_μμ w.r.t. r_i.
    Accumulate F_i^(k-space).
end for
for all i do
    Add self-term corrections:
        F_i += F_i^(self).
end for
```

17 How to Cite This Code

When using the CCpol23+ MD Package in published work, please cite the Zenodo record associated with this software [4], for example:

> O. Ishaque, *Molecular Dynamics Simulation package for the Rigid CCpol23+ Water Model Using Quaternion-Based Rigid Body Dynamics*, Zenodo (2025), DOI: `[https://doi.org/10.5281/zenodo.14968189]`.

Acknowledgements

Development of this code has been carried out as part of research on first-principles-based water models and many-body interactions. The author gratefully acknowledges collaborators and funding sources that have supported this work, as well as the availability of high-performance computing resources used for testing and validation.

References

[1] U. Góra, W. Cencek, R. Podeszwa, A. van der Avoird, and K. Szalewicz. Predictions for water clusters from a first-principles two- and three-body force field. *J. Chem. Phys.*, 140:194101, 2014.

[2] T. F. Miller, M. Eleftheriou, P. Pattnaik, A. Ndirango, D. Newns, and G. J. Martyna. Symplectic quaternion scheme for biophysical molecular dynamics. *J. Chem. Phys.*, 116:8649, 2002.

[3] W. Smith, T.R. Forester, and I.T. Todorov. *DLPOLY Classic User Manual 1.10, STFC Daresbury Laboratory Daresbury*, 2017.

[4] Ommair Ishaque. Molecular dynamics simulation package for the rigid CCpol23+ water model using quaternion-based rigid body dynamics. *Zenodo*, 2025. https://doi.org/10.5281/zenodo.17821225.